Analyze this codebase as if you are a principal architect reviewing an AI platform.

Your task is to infer the actual system architecture from the implementation, not from assumptions.

Explain:

1. The major subsystems/services present in the codebase

2. The responsibility of each subsystem

3. How requests flow through the system end-to-end

4. Which parts are tightly coupled and which are modular

5. What architectural style this resembles (monolith, modular monolith, microservices, service-oriented, hybrid)

6. Where the architecture is clean and where it is likely to break at scale

Return the output as:

- Executive summary

- Architecture breakdown

- End-to-end request lifecycle

- Design risks

- Recommended refactor priorities

Codebase:

Executive Summary

JoaLLM is a modular monolith — a single deployable process on Railway with internal domain separation that partially mimics service boundaries, but shares one database, one Redis instance, one process space, and one deployment unit. The architecture is well-structured for a seed-stage product: fast to iterate, easy to deploy, and coherent enough to not trip over itself. The main risks are in the document pipeline (single-process workers blocking the web server), missing abstractions where business logic has leaked into route handlers, and two overlapping routing systems that will cause confusion as the team grows.

Architecture Breakdown

1. API Gateway / HTTP Layer

Files: src/index.ts, src/domains/index.ts, src/middleware/

Fastify is bootstrapped with plugins registered at startup: CORS, multipart, rate limiting (global 100 req/min), swagger. All domains are registered sequentially via registerDomains(). Security headers applied via a hook.

The middleware stack is:
Request → monitoring → auth (JWT verify + blacklist check) → requireTier/requireRole → route handler

Critical detail: rate limiting is global, not per-user or per-route. One heavy user can consume headroom for everyone at 100 req/min flat.

---

2. Domain Layer (15 domains)

Files: src/domains/*/index.ts

Domains: auth, chat, files, rag, models, workflows, notebooks, bookmarks, preferences, security, telemetry, survey, policy, user-settings, health.

Each domain registers routes under a prefix. The domain files are thin — most just import from src/routes/ and re-export. The actual business logic lives in src/routes/*.ts, not in the domain files. So the domain layer is organizational, not encapsulating — it draws a boundary in the file tree but not in execution.

---

3. Route Handlers (heavy business logic)

Files: src/routes/*.ts

This is where most logic lives. Each route handler directly:
- Validates the request body (Zod inline or jsonSchema)
- Queries the DB via Drizzle
- Calls services
- Assembles the response
- Writes to audit log

rag.ts is the most complex — ~1,400 lines handling search, chat, session management, multi-hop retrieval, LLM routing, confidence evaluation, and usage logging all in one file.

---

4. Document Ingestion Pipeline

Files: src/services/queue.ts, document-processor.ts, embedding-service.ts, adaptive-chunker.ts, file-storage.ts

Two-queue BullMQ pipeline backed by Redis:

HTTP upload → files table (status: pending)
→ documentProcessingQueue
└─► Worker 1: extract text, chunk (adaptive-chunker), store to R2/S3/volume
→ documentIndexingQueue
└─► Worker 2: generate embeddings, upsert document_chunks with vector

Both workers run inside the same Fastify process. Redis is optional — if unavailable, uploads fail silently (the warning says "processed synchronously" but there's no sync fallback path in the upload route).

EmbeddingService uses Cohere as primary, OpenAI text-embedding-3-small as fallback. Same new EmbeddingService(userApiKeys) per-call pattern as the LLM service (not yet cached).

---

5. RAG Retrieval Layer

Files: enhanced-rag-service.ts, rag-service.ts, rag-modes.ts

enhanced-rag-service.ts is the production path. rag-service.ts still exists as a legacy path (used by the indexing worker and some older routes). Two implementations of similar retrieval logic — not fully consolidated.

Retrieval: pgvector cosine distance + FTS ts_rank merged as weighted hybrid. Mode configs (rag-modes.ts) parameterize everything: k, threshold, weights, temperature, maxTokens, system prompt builder.

---

6. LLM Provider Layer

Files: src/services/llm-providers.ts

LLMService wraps OpenAI, Anthropic, Groq, Ollama, and MockProvider. Provider selected by model name prefix. User API keys override system keys. SDK clients now cached per key fingerprint (just fixed). No streaming in RAG path.

---

7. Access Control / Entitlements

Files: middleware/auth.ts, middleware/subscription.ts, services/access-control.ts

Three-layer gate:
- Auth: JWT verify + Redis blacklist check per request
- RBAC: role → permission matrix (casual < premium < admin < superuser)
- Tier: subscription limits (free < pro < enterprise) enforced per route via requireTier() middleware

requireTier() hits the DB on every gated request — no cache. At volume this is a per-request DB round-trip for every premium feature.

---

8. Cache Layer

Files: src/services/cache.ts

Redis-backed with three defined key namespaces: user:profile:*, models:list, rag:search:*. The cache is optional and degrades gracefully. Currently only RAG search results and model lists are cached. User API keys, tier lookups, and embedding calls are not.

---

9. Observability

Files: src/middleware/monitoring.ts, src/utils/prometheus-metrics.ts, src/utils/audit.ts, src/utils/logger.ts

Prometheus metrics exposed at /metrics. Audit log writes to DB (audit_logs table) for security-sensitive actions. Pino logger. No distributed tracing (no trace IDs correlating across the document pipeline).

End-to-End Request Lifecycle

RAG Chat (POST /api/rag/chat)

1. Fastify receives request
2. monitoring middleware records request start
3. optionalAuth: verifies JWT, checks Redis blacklist, sets request.user
4. Route handler begins:
   a. Parse + validate body (message, documentIds, mode, model)
   b. getMode() → RAGModeConfig (params + prompt builders)
   c. DB: ownership check on documentIds (new — just added)
   d. DB: getUserApiKeys (decrypt user's stored provider keys)
   e. enhancedRAGService.searchWithConfidence():
      - embed query (Cohere → OpenAI fallback)
      - pgvector ANN search
      - FTS search
      - merge + score + threshold filter
      - confidence tier (or bypass if explicit file selection)
   f. If Research mode: second embedding + retrieval pass (sequential)
   g. formatContext() → string
   h. buildSystemPrompt() + buildUserPrompt() → messages[]
   i. LLMService.getProvider(model) → provider
   j. provider.generateResponse(messages, model, params) → LLM API call
   k. DB: log to api_usage
   l. Assemble response: { response, sources, confidence, mode, usage }
5. Reply sent
6. monitoring middleware records request end + Prometheus counters

Document Upload (POST /api/files/upload)

1. Auth + tier check (maxFiles, maxStorageMB limits verified via DB)
2. Multipart file read into memory buffer
3. DB insert: files row (status: pending)
4. documentProcessingQueue.add(job) → Redis
5. Reply 201 immediately

[Background - Worker 1]

6. Dequeue job
7. Buffer → documentProcessor.processDocument() (text extraction)
8. Optional: storageProvider.uploadFile() (R2/S3/volume)
9. adaptive-chunker → chunks[]
10. DB update: files (status, storageUrl, metadata)
11. documentIndexingQueue.add({ fileId })

[Background - Worker 2]

12. Dequeue indexing job
13. DB: fetch chunks for fileId
14. embeddingService.generateEmbedding() per chunk
15. DB upsert: document_chunks with vector
16. DB update: files (status: indexed)

Design Risks

1. Workers co-located with the web server

Both BullMQ workers boot inside index.ts. Heavy document processing (PDF parsing, chunking, embedding API calls) competes for CPU and memory with live request handling. Under upload load, p99 latency on chat routes will spike. Breaks at: sustained multi-user concurrent uploads.

2. rag.ts is a 1,400-line god file

All RAG orchestration, retrieval configuration, session tracking, LLM routing, and usage accounting live in one route file. Adding a new mode, a new retrieval strategy, or a new LLM behavior requires editing this file. Test surface is enormous and currently untested (no unit tests for this route). Breaks at: second engineer on this codebase,
or any significant RAG feature work.

3. Two retrieval implementations

rag-service.ts and enhanced-rag-service.ts both exist. The legacy service is still called by the indexing worker and some routes. They will diverge silently — bug fixed in one, not the other. Breaks at: any retrieval correctness work.

4. requireTier() hits DB on every gated request

No caching. Each call to a tier-gated route makes a SELECT subscriptionTier FROM users WHERE id = ?. At 2,000 req/day per pro user this is tolerable, but at multi-tenant load with many gated routes it becomes a consistent query pattern with no benefit. Breaks at: 1,000+ concurrent active users.

5. RAG chat has no streaming

/api/rag/chat blocks until the full LLM response is generated. Research mode with maxTokens=2500 means users wait 5–15 seconds with no feedback. Breaks at: user retention, especially for Research and Decision modes.

6. EmbeddingService instantiated per call (same pattern as LLM, not yet fixed)

generateEmbedding() creates new EmbeddingService(userApiKeys) per invocation — new Cohere and OpenAI SDK clients each time. The LLM cache was just fixed; this one wasn't. Breaks at: high-frequency RAG usage.

7. No token budget enforcement in context assembly

formatContext() concatenates all retrieved chunks with no awareness of the target model's context window. Research mode returning 25 chunks could silently overflow context. Breaks at: any model with a 4k or 8k context, and degrades quality on 32k models with very long documents.

8. Redis unavailability silently disables uploads

When Redis is down, documentProcessingQueue.add() throws. The upload route returns 500 but the files row is already committed (status: pending, never updated). These orphaned rows accumulate invisibly. Breaks at: any Redis flap in production.

Recommended Refactor Priorities

P0 — Correctness

- Consolidate rag-service.ts into enhanced-rag-service.ts and delete the legacy file. One retrieval implementation.
- Add a transactional guard around the upload route: if queue enqueue fails, roll back the files insert (or move the insert into Worker 1 as the first step).

P1 — Scalability ceiling

- Extract BullMQ workers into a separate process/Railway service. The API server should only enqueue; workers run independently. This is the single highest-leverage architectural change available.
- Cache requireTier() and getUserApiKeys() results in Redis with a 5-minute TTL. Both are called on nearly every authenticated request and are stable data.

P2 — Developer velocity

- Extract a RAGOrchestrator class from rag.ts. The route handler should call orchestrator.chat(request) and nothing else. All mode selection, retrieval, prompt assembly, LLM dispatch, and usage logging live in the orchestrator. rag.ts becomes ~100 lines.
- Add streaming to /api/rag/chat using the same SSE pattern as /api/chat/stream.

P3 — Correctness under load

- Add token budget enforcement in formatContext(): estimate chunk tokens, accumulate, stop at modelContextLimit - systemPromptTokens - maxTokens.
- Cache EmbeddingService instances by API key fingerprint (same pattern just applied to LLMService).
- Add per-user rate limiting as a separate tier from the global 100 req/min Fastify limit.

Review the repository structure and infer the design philosophy behind the platform.

Based on folders, modules, services, configs, models, routes, workers, and frontend structure, explain:

- What kind of platform this is trying to become

- Whether the repository structure supports that goal

- Where structure reflects good separation of concerns

- Where structure suggests accidental complexity

- Which folders/components indicate future extensibility

Also identify:

- Dead-looking patterns

- Misleading naming

- Structural inconsistencies

- Missing abstraction layers

Codebase:

What Kind of Platform This Is Trying to Become

The intent is clear and ambitious: an enterprise AI workspace — a single product where a team can do everything LLM-related without leaving. The feature surface confirms this:

- RAG knowledge base with multi-mode retrieval
- Multi-provider LLM chat (OpenAI, Anthropic, Groq, Cohere, Ollama)
- Notebook environment with JS sandbox and AI cells
- Visual workflow builder (ReactFlow-based DAG executor)
- RBAC + subscription tier enforcement
- 2FA, audit logging, token blacklisting
- Prometheus/Grafana observability stack
- BYOK (Bring Your Own Key) for Pro/Enterprise
- Feedback + training signal capture

The positioning is "AI Swiss Army Knife" — Notion meets LangChain meets a RAG platform, sold horizontally across team sizes from free solo users to enterprise contracts.

Whether the Structure Supports That Goal

Mostly yes, with one critical gap.

The domain decomposition, tiered access control, pluggable LLM providers, and queue-based document pipeline are all correct structural choices for this product. The monorepo with three services (backend, frontend, landing-page) is appropriate at this stage.

The critical gap: the platform is trying to become multi-tenant and enterprise-grade, but the structure is still fundamentally single-tenant. There is no workspace or organization table, no tenant isolation in data queries, no per-tenant configuration surface. Every feature references userId as the root identity. Adding real multi-tenancy
later will require touching nearly every route and query — the longer this waits, the more expensive it becomes.

Where Structure Reflects Good Separation of Concerns

Access control is cleanly layered.
Three distinct files, three distinct concerns: auth.ts (identity verification), subscription.ts (entitlement limits), access-control.ts (RBAC permissions). They compose without bleeding into each other. The requireTier() and requirePermission() factories are usable independently at route registration time. This is the best-designed subsystem
in the codebase.

LLM provider abstraction is correct.

Each provider (OpenAIProvider, AnthropicProvider, GroqProvider, OllamaProvider) implements the same interface. LLMService.getProvider() routes by model prefix. New providers can be added without touching call sites. The MockProvider for offline dev is a thoughtful touch.

RAG mode configs are well-isolated.

rag-modes.ts owns all retrieval and generation parameters per mode. Adding a fifth mode requires editing exactly one file. The prompt builders are co-located with the parameters they control. This is the right design.

BullMQ two-queue pipeline.

Separating text extraction (queue 1) from embedding generation (queue 2) is architecturally sound — they have different failure profiles, different retry needs, and different resource consumption. The design is right even if the execution (same process) is currently wrong.

Frontend service layer.

src/services/*.ts files are thin API clients — they do not own state, do not contain business logic, and map 1:1 to backend domain areas. This is the correct pattern. Hooks compose over services; components compose over hooks. The layering holds.

Drizzle schema is the single source of truth.

All table definitions, relations, indexes, and type inference live in one file (schema.ts). Migrations are tracked. The pgvector extension is integrated cleanly via the Drizzle adapter.

Where Structure Suggests Accidental Complexity

The domain layer is a naming layer, not a code layer.

Every src/domains/*/index.ts is 10–20 lines that imports from src/routes/ and calls fastify.register(). The domains draw a box in the file tree but all the logic is in routes. This creates a navigation indirection — you read domains/rag/index.ts to discover you need to look at routes/rag.ts — with no encapsulation benefit. Either the domains should own their logic (true DDD) or be deleted and routes registered directly.

Two parallel RAG implementations.

rag-service.ts and enhanced-rag-service.ts both implement hybrid retrieval. The legacy one is still called by the indexing worker and some routes. They will diverge silently. This is not a feature — it's a refactor that was started and not finished.

The landing page is a full application clone.

The landing page service contains full copies of components/chat/, components/rag/, components/knowledge/, components/workflow/, contexts/, hooks/, services/, stores/, and utils/ — identical to the frontend. None of these are used by the marketing page. They appear to have been copied wholesale when the landing page was scaffolded. This is ~60% of the landing page src/ tree being dead weight.

shared/ is structurally present but empty in practice.

The repo has shared/sdk/, shared/events/, shared/types/, shared/services/, shared/utils/, shared/config/. The domain-events.ts inside the backend suggests an event system was being designed. But nothing in the frontend or landing page imports from shared/. It is infrastructure for a contract that was never published.

Two Redis clients in the same process.

queue.ts uses ioredis for BullMQ. cache.ts uses the redis npm package (v4 client) for caching. Same Redis instance, two different client libraries, two separate connections from the same process. This is an accidental divergence — BullMQ required ioredis, the cache was added later with the newer redis package, and they were never consolidated.

Four RAG-related frontend hooks.

useRAG, useAdvancedRAG, useRAGChat, and useRAGChatSessions all exist independently. There is also useRAGSessions and useRAGSuggestions. Six hooks for one feature domain. Some are wrappers around others, some are parallel implementations. The same fragmentation appears in services: ragApi.ts, ragChatApi.ts, ragChatSessionsApi.ts, ragSessionApi.ts — four files for one API domain.

lint: --max-warnings=350.
The frontend lint script allows up to 350 warnings before failing. This is a sign that lint debt was allowed to accumulate and then suppressed rather than fixed. It means the lint gate provides almost no signal.

Which Components Indicate Future Extensibility

domain-events.ts + shared/events/
The event bus infrastructure (createDomainEvent, emitDomainEvent, registerDomainEventDispatcher) and the shared/sdk/src/events/ directory signal intent to move toward event-driven inter-service communication. The five defined event types (chat.session.created, rag.document.indexed, etc.) are exactly the right events for a platform that will
need webhooks, audit streams, or downstream ML pipelines.

shared/ directory
Even though currently empty, its existence with subdirectories sdk/, types/, events/, services/ is a deliberate architectural statement: this will become a shared contract layer when the frontend and backend need to share types, or when a third service (e.g., a standalone worker service) is extracted.

Notebook cell types
The CellSchema enum — markdown | code | ai | chart | knowledge | agent | debug — is more ambitious than what's currently built. agent and debug cell types suggest planned agentic notebook execution. The cell type vocabulary is designed for a product 6–12 months ahead of the current implementation.

docker-compose.microservices.yml
A separate compose file for microservices topology already exists alongside the monolith compose. This is the extraction path being kept warm. When workers are split out, the compose file is the deployment blueprint.

Workflow executor DAG engine
The workflow-executor.ts is a general-purpose DAG runner with typed node interfaces (WorkflowNode, WorkflowEdge). It is not coupled to any specific node type — new node types are added by extending executeNode()'s switch statement. The parallel execution rewrite makes it production-capable. This is the right kernel for a future agent
orchestration layer.

TOTP + token blacklisting + audit log

The security primitives (TOTP 2FA via otplib, token blacklisting in Redis, typed audit log with action enum) are enterprise-grade infrastructure that most early-stage platforms skip. Their presence signals that the enterprise tier is being built seriously, not as an afterthought.

Dead-Looking Patterns

Pattern	Location	Evidence of being dead
ChatInterfaceNew.tsx	frontend/src/components/chat/	Sibling to ChatInterface.tsx — a migration that was never finished or cleaned up
KnowledgeManagerNew.tsx	frontend/src/components/knowledge/	Same pattern — parallel "New" component alongside original
UserRoleContext.tsx	frontend/src/contexts/	Shadowed by EnhancedUserRoleContext.tsx in the same folder
useRAG.ts	frontend/src/hooks/	Likely superseded by useAdvancedRAG.ts and useRAGChat.ts
rag-service.ts	backend/src/services/	Superseded by enhanced-rag-service.ts
Root-level .js scripts	/clear-storage-cli.js, /test-llm-service.js, test-oauth*.html	Debugging scripts committed to the root, no longer maintained
monitoring/data/prometheus/	Root monitoring/ directory	Data directory for a locally-run Prometheus — committed to the repo
FeaturesOverview.tsx	landing-page/src/components/	Not imported in App.tsx; sibling to Features.tsx
AIInsights.tsx	landing-page/src/components/	Not imported in App.tsx
Survey.tsx / SurveyModal.tsx	landing-page/src/components/	Not in App.tsx; likely removed from the page but not from the tree

---

Misleading Naming

domains/ implies DDD; delivers routing registration.

In DDD, a domain owns its logic, its types, its persistence. Here, domains are registration wrappers. Calling the folder domains/ creates an expectation the code does not fulfill. A more honest name would be routes/registry/ or the domain files could be deleted.

googleAuthService.ts vs authService.ts (frontend)

Backend has googleAuthService.ts as a singleton service. Frontend has authService.ts which handles both email/password and Google OAuth. The naming asymmetry makes it hard to reason about which authentication flows live where.

local-file-storage.ts vs file-storage.ts

file-storage.ts exports a storageProvider that routes between R2, S3, and local. local-file-storage.ts is the local implementation. But file-storage.ts is named as if it's the local provider. The abstraction file should be named storage-provider.ts or file-storage-factory.ts.

JoaLLMFarm.tsx

Located at components/farm/JoaLLMFarm.tsx in both frontend and landing page. The name is opaque. Without reading the file, there is no way to know what this is. The farm/ folder has no conventional meaning in a React application.

me.ts route

The /api/me route is named like a user profile endpoint but based on its registration context (it handles "access bootstrap") it is actually an entitlements/permissions summary endpoint. /api/me/access would be more descriptive than a top-level me.ts.

Structural Inconsistencies

Four admin/management routes not going through the domain layer.
admin.ts, subscriptions.ts, me.ts, and feedback.ts are registered directly in domains/index.ts via fastify.register() rather than through a domain wrapper. Every other route has a domains/*/index.ts file. These four are exceptions with no structural explanation.

Backend has winston AND pino as dependencies.
fastify uses Pino internally. logger.ts likely wraps one of them. winston is in package.json as a dependency. Having both is either a leftover migration or a case where different parts of the codebase use different loggers — which means logs flow to different sinks with different formats.

Frontend has both Zustand stores AND TanStack Query.
Three Zustand stores (chatStore, ragChatStore, ragSessionStore) handle server state that TanStack Query is also managing via hooks. These are two competing server-state management strategies in the same app. TanStack Query should own server state; Zustand should own local UI state. The current setup creates ambiguity about which is the authoritative source of truth for chat and RAG session data.

types/index.ts is monolithic on the frontend.
The entire type surface of the frontend lives in one file. Meanwhile the backend has a proper types/ directory with error.d.ts, request.d.ts, and responses.ts. The frontend pattern will not scale — a single index.ts for all types across auth, chat, RAG, notebooks, workflows, and documents becomes unmaintainable.

Missing Abstraction Layers

No RAGOrchestrator.
The most complex coordination logic in the system lives inline in a 1,400-line route file. A RAGOrchestrator class would give this logic a home, a test surface, and a stable interface.

No Repository layer.
Every route handler queries the database directly via Drizzle. There is no UserRepository, FileRepository, or DocumentRepository. This means the same query (SELECT FROM files WHERE userId = ? AND status = ?) is written in multiple routes with slight variations. A repository layer would centralize data access, make queries testable without an HTTP server, and make it possible to change the ORM without touching routes.

No request/response DTOs.
types/responses.ts exists on the backend but is minimal. There is no systematic DTO layer mapping between database rows and API responses. Routes return Drizzle row shapes

directly. When the schema changes, the API shape changes. When you need to hide a field from the API, you have to remember to strip it in every route that returns that entity.

No tenant/workspace identity.

As noted, userId is the root identity everywhere. There is no Tenant, Organization, or Workspace entity despite the enterprise positioning. This is the missing abstraction that will require the most invasive refactor when multi-tenancy becomes necessary.

No event bus in use.

domain-events.ts built the infrastructure. The shared/events/ directory exists. But emitDomainEvent() is not called anywhere in the routes or services. The event system was built but never wired up — it is an abstraction waiting for adoption.

Inspect the codebase and identify all logical services/modules.

For each service/module, provide:

- Purpose

- Inputs

- Outputs

- Dependencies

- Whether the boundary is clean or blurred

- Whether it should remain independent or be merged/refactored

Then answer:

- Are current service boundaries aligned with an AI platform?

- Which services should exist but are currently missing?

- Which services are overloaded with too many responsibilities?